Les Mathématiques du Pong : Un Cours Illustré 🏓

Introduction : Pourquoi des maths dans un jeu ?

Imaginez une balle de ping-pong qui rebondit sur votre table. Pour la programmer dans un jeu vidéo, il faut traduire ce mouvement physique en mathématiques ! Ce cours explore les concepts mathématiques cachés derrière un simple jeu de Pong.

---

1. Les Vecteurs : La Direction et la Vitesse 📐

Qu'est-ce qu'un vecteur ?

Un **vecteur** est comme une flèche qui indique :

- 📍 **Une direction** (où va la balle ?)
- 📏 **Une intensité** (à quelle vitesse ?)

Dans notre jeu Pong, la vitesse de la balle est représentée par un vecteur avec deux composantes :

- `balldirX` : vitesse horizontale (gauche/droite)
- `balldirY` : vitesse verticale (haut/bas)

Exemple concret 🎯

Si `balldirX = 6` et `balldirY = 8` :

- La balle se déplace de 6 pixels vers la droite ET 8 pixels vers le haut à chaque frame
- C'est comme si vous marchiez 6 pas vers l'est et 8 pas vers le nord

```
↑ balldirY = 8
|
|
+---→ balldirX = 6
```

---

2. Le Théorème de Pythagore : Calculer la Vitesse Totale 📊

Le principe

Même si la balle se déplace en diagonale, on peut calculer sa vitesse totale grâce au **théorème de Pythagore** :

Vitesse totale = √(vitesseX² + vitesseY²)

Analogie : La Marche en Ville 🚶

Imaginez que vous marchez dans une ville :

- Vous faites 6 pâtés de maison vers l'est
- Puis 8 pâtés de maison vers le nord
- **Distance parcourue** : 6 + 8 = 14 pâtés de maisons
- **Distance réelle** (à vol d'oiseau) : $\sqrt{(6^2 + 8^2)} = \sqrt{(36 + 64)} = \sqrt{100} =$ **10 pâtés**

C'est exactement pareil pour notre balle !

Dans le code

```javascript
const currentSpeed = Math.sqrt(balldirX * balldirX + balldirY * balldirY);
```

---

3. Les Angles et les Fonctions Trigonométriques 📐

Le Cercle Trigonométrique

Notre jeu utilise les fonctions **cosinus** et **sinus** pour créer des directions aléatoires :

```javascript
balldirX = ballSpeed * Math.cos(angle);
balldirY = ballSpeed * Math.sin(angle);
```

Analogie : L'Horloge Magique ⏰

Imaginez une horloge géante où :

- L'aiguille a toujours la même longueur (`ballSpeed`)
- Selon où pointe l'aiguille, on obtient différentes directions

Cosinus = composante horizontale (gauche/droite) **Sinus** = composante verticale (haut/bas)

Exemples pratiques

Angle	Direction	cos(angle)	sin(angle)	Résultat
0°	→ Droite	1	0	Tout horizontal
90°	↑ Haut	0	1	Tout vertical
180°	← Gauche	-1	0	Tout horizontal (inverse)
45°	↗ Diagonale	0.7	0.7	Équilibré

Pourquoi certains angles sont interdits ?

Le jeu évite les angles trop verticaux ou horizontaux car :

- **Trop vertical** : la balle monte/descend sans fin → ennuyeux !
- **Trop horizontal** : la balle va trop vite à gauche/droite → impossible à rattraper !

---

4. La Normalisation : Garder une Vitesse Constante ⚖️

Le Problème

Après plusieurs rebonds, la vitesse peut légèrement changer à cause des imprécisions de calcul. C'est comme si votre voiture accélérait ou ralentissait toute seule !

La Solution : Le Vecteur Unitaire

Un **vecteur unitaire** est un vecteur de longueur exactement 1. C'est comme avoir une flèche qui indique juste la direction, sans se soucier de la vitesse.

Analogie : Le GPS qui Recalibre 🧭

Imaginez que votre GPS dit :

1. "Vous allez dans la bonne direction, mais trop vite"
2. Il calcule votre direction exacte
3. Il vous dit : "Ralentissez à exactement 50 km/h, mais gardez la même direction"

C'est exactement ce que fait la normalisation !

Étapes de la normalisation

```javascript
// 1. Calculer la vitesse actuelle
const currentSpeed = Math.sqrt(balldirX * balldirX + balldirY * balldirY);

// 2. Créer un vecteur unitaire (longueur = 1)
const unitX = balldirX / currentSpeed;  // Direction X pure
const unitY = balldirY / currentSpeed;  // Direction Y pure

// 3. Remettre à la bonne vitesse
balldirX = unitX * ballSpeed;  // Direction X × vitesse voulue
balldirY = unitY * ballSpeed;  // Direction Y × vitesse voulue
```

Exemple Numérique 🔢

Situation : La balle a `balldirX = 6`, `balldirY = 8`, mais on veut `ballSpeed = 9`

1. **Vitesse actuelle :** $\sqrt{(6^2 + 8^2)} = \sqrt{100} = 10$ ❌ (trop rapide !)
2. **Vecteur unitaire :**
   - `unitX = 6/10 = 0.6`
   - `unitY = 8/10 = 0.8`
   - Vérification : $\sqrt{(0.6^2 + 0.8^2)} = \sqrt{(0.36 + 0.64)} = \sqrt{1} = 1$ ✅
3. **Renormalisation :**
   - `balldirX = 0.6 × 9 = 5.4`
   - `balldirY = 0.8 × 9 = 7.2`
   - Vérification : $\sqrt{(5.4^2 + 7.2^2)} = \sqrt{(29.16 + 51.84)} = \sqrt{81} = 9$ ✅

---

5. Les Rebonds : Physique Simplicifiée 🎱

Rebond sur un Mur Horizontal

Quand la balle touche le plafond ou le sol :

```javascript
balldirY *= -1;  // Inverser la direction verticale
```

Analogie : C'est comme faire rebondir une balle de tennis contre un mur. La vitesse horizontale ne change pas, mais la vitesse verticale s'inverse.

Rebond sur une Raquette

Même principe, mais on inverse la direction horizontale :

```javascript
balldirX *= -1;  // Inverser la direction horizontale
```

---

6. Mise en Pratique : Le Debug Visuel 🔍

Le jeu affiche en temps réel :

- La vitesse actuelle calculée
- Si elle correspond à la vitesse voulue (9)
- Un indicateur vert ✅ ou rouge ❌

C'est comme avoir un compteur de vitesse dans votre voiture qui vous dit si vous respectez la limitation !

Conclusion : Des Maths Partout ! 🎮

Ce simple jeu de Pong utilise :

- **Géométrie** : vecteurs, angles, distances
- **Trigonométrie** : cosinus, sinus
- **Algèbre** : normalisation, calculs de vitesse
- **Physique** : rebonds, conservation de l'énergie

Ces concepts se retrouvent dans tous les jeux vidéo modernes, de Pac-Man à Call of Duty ! Les mathématiques sont vraiment la langue secrète des jeux vidéo. 🚀

Pour Aller Plus Loin 📚

- **Vecteurs 3D** : ajouter une dimension Z pour les jeux 3D
- **Accélération** : changer la vitesse progressivement
- **Collision avancée** : détection précise des formes complexes
- **IA** : faire bouger les raquettes automatiquement